

Reimplementação GPU do pipeline mbkmeans (scRNA-seq)

Resumo técnico para apresentação—pré-processamento,clusteringinicial,normalização, PCA e clustering final

Paper de referência: Hicks et al. (2021), PLOS Comput. Biol. — mini-batch k-means on-disk

1 — A lógica geral do trabalho

O objetivo foi reimplementar em **GPU (CuPy/CUDA) + CPU multicore (Numba/OpenMP)** um pipeline de análise de scRNA-seq que o paper original executa em **CPU single-thread (R/C++)**. O dado é uma matriz **esparsa** de ~1,3 milhão de células × ~28 mil genes, armazenada em HDF5. Cinco passos foram reimplementados:

- **Pré-processamento (QC/filtragem)** — equivalente ao scater do paper.
- **Clustering inicial (mini-batch k-means)** — serve de entrada para a normalização scan; não é o resultado final, é um passo intermediário.
- **Normalização (scan/deconvolution)** — o passo que garante o sinal biológico antes de qualquer análise; usa o clustering inicial.
- **PCA (50 componentes)** — redução de dimensionalidade; reformulada por covariância.
- **Clustering final (K-Means completo via Lloyd, pós-PCA)** — o resultado que o paper de fato reporta e valida biologicamente; aqui vamos além dele com busca automática de K.

A tese do trabalho é paralelizarmos as operações do paper, procurando maior velocidade mas abrimos o estudo para analisar coisas essenciais, como: "GPU é sempre mais rápida?". Fizemos isso com medições, **onde** a GPU ajuda, **onde** a CPU é a escolha certa, e **por quê** — uma análise de trade-off de arquitetura que o paper (CPU-only) não faz.

2 — A regra central: a GPU só compensa quando o gargalo é cálculo

Uma GPU é poderosa no **cálculo**, mas o dado precisa **viajar até ela: SSD → RAM → VRAM → cálculo**. Se o tempo é gasto **movendo dado** (ler do disco ou transferir pela PCIe), acelerar o cálculo quase não muda o tempo total — a GPU fica **ociosa esperando dado**. Ela só rende quando o cálculo é **pesado, regular e reutiliza** os dados já transferidos.

Passo	Onde está o gargalo	Natureza	GPU compensa?
Pré-proc. (QC)	SSD → RAM (I/O de disco)	I/O-bound, cálculo leve	Pouco
Clustering inicial	RAM → VRAM (PCIe, repetido)	Transfer-bound + esparsa	Não compensa
Normalização	Cálculo por bloco denso	Compute-bound, bounded	Sim
PCA	Nenhum — é o cálculo	Compute-bound, denso	Sim, muito
Clustering final	Nenhum — dado cabe na VRAM	Compute-bound, denso (220 MB)	Sim, decisivo

Analogia: uma Ferrari (a GPU) presa num engarrafamento (I/O ou PCIe) anda na velocidade do trânsito — trocar o motor não adianta; o que limita é a estrada.

3 — Pré-processamento (QC): GPU no Pass 1, CPU no Pass 2

O que faz: varre o HDF5 em dois passes. O Pass 1 calcula as métricas de QC e decide quais células/genes ficam; o Pass 2 monta a matriz filtrada num buffer pré-alocado.

Pass 1 → GPU (porque é CÁLCULO regular). As métricas de QC (contagem por célula, % mitocondrial) são calculadas com prefix sums (cumsum) direto sobre os arrays CSR, sem nunca densificar a matriz. O acesso é sequencial e regular (varredura linear), padrão onde a GPU é eficiente. Aqui está o ganho legítimo.

Pass 2 → CPU multicore (porque é MOVIMENTAÇÃO de memória). Montar a matriz filtrada é copiar e remontar dados, não calcular. Feito na CPU com Numba + OpenMP (@njit(parallel=True)),

paralelizando por célula. Por que não GPU?

- Levar à GPU só pra copiar pagaria PCIe ida e volta por uma operação sem cálculo.
- A escrita é irregular (cada célula tem nº diferente de não-zeros) — GPU odeia isso.
- É embaraçosamente paralelo por célula, sem race condition (ranges não se sobrepõem) — cenário ideal pra CPU multicore.

Por que o ganho foi modesto (8 min → 6 min, ~25%): o passo é I/O-bound — o tempo é dominado por ler 2,4 bilhões de não-zeros do SSD, não por calcular. O disco não fica mais rápido só porque quem processa é a GPU.

4 — Clustering inicial: roda em GPU, mas a GPU não compensa aqui

Importante para a banca: o clustering é implementado em GPU (CuPy) — assignment, cluster-sums e update dos centroides rodam na VRAM, com os centroides residentes na GPU. Mesmo assim, a GPU não traz ganho expressivo neste passo. Isso **não é defeito — é um resultado medido**. Três fatores se somam:

Fator	O que acontece
Iterações repetidas	Cada uma das ~2.000 iterações reenvia um batch novo pela PCIe. O QC transfere 1 vez; o clustering, milhares.
Cálculo barato demais	A GPU calcula as distâncias em microssegundos, mas a transferência custou milissegundos. Ela termina e fica ociosa esperando o próximo batch (“data starvation”).
Esparsidade	Em CSR, a thread precisa olhar o índice antes de buscar o valor (indireção). Isso quebra o acesso coalescido — threads vizinhas leem posições espalhadas — e desperdiça a largura de banda da GPU.

As métricas reforçam: do tempo total do clustering, uma fatia grande é I/O e varredura final (assignment final + salvar ≈ 168 s), não o algoritmo em si. Tanto no QC quanto no clustering, muito do tempo é mover dado — e é por isso que a GPU tem pouco espaço para brilhar.

5 — Frases-âncora: pré-processamento e clustering inicial

Sobre o ganho variar entre passos:

“O ganho varia porque cada passo tem um gargalo diferente: o QC é I/O-bound (lê do disco), o clustering é transfer-bound (PCIe + esparsos), e o PCA é compute-bound. A mesma GPU dá ~25% num caso e ordens de magnitude no outro — saber distinguir isso é o cerne do trabalho.”

Sobre usar GPU no clustering:

“O clustering é implementado em GPU, mas medimos que aqui ela não compensa: o gargalo é a transferência PCIe repetida de dados esparsos com acesso irregular, não o cálculo. O ganho real está no pré-processamento. Mostrar onde a GPU ajuda e onde não ajuda é parte da nossa contribuição.”

Sobre o ganho modesto no pré-processamento:

“O ganho no QC é modesto porque o passo é dominado por I/O de disco — acelerar a GPU não muda um tempo que é gasto esperando o SSD. A GPU acelera o cálculo, mas o cálculo já não era o gargalo.”

6 — Normalização (scrn): reimplementação fiel do algoritmo na GPU

Ponto que a banca cobra: o paper não descreve normalização — ele usa o scrn como peça pronta (Lun, Bach & Marioni, 2016). Logo a referência do nosso código não é o paper mbkmeans, e sim o algoritmo de deconvolution do scrn. A frase correta é: “o paper terceiriza a normalização pro scrn; eu reimplementei o scrn na GPU”.

Por que scran e não normalização simples (CPM/library-size): a library-size assume que toda célula tem a mesma composição de RNA. Se um tipo celular expressa mais RNA por razão biológica real, a CPM trata isso como artefato técnico e “corrige” — apagando o sinal. O scran compara cada célula com células parecidas (pooling), produzindo um fator por célula que respeita as diferenças biológicas. É daí que vem a dependência: o pooling só é válido entre células homogêneas, então o scran exige um clustering inicial — exatamente o passo 4. Por isso o clustering com todos os genes existe: é insumo do scran.

O esqueleto do código é scran puro — segue a receita passo a passo:

Passo do scran (Lun 2016)	No código	Fidelidade
Pré-escala por library size	$t = \text{lib}/\text{média}$; $Z = \text{counts}/t$	Idêntico
Ordenar células por library size	<code>argsort(lib)</code>	Idêntico
Pseudo-célula de referência (média)	<code>ref = Zo.mean(0)</code>	Idêntico
Filtro de genes (min.mean=0.1, UMI)	<code>mask = ref ≥ 0.1</code>	Idêntico (default)
Pools circulares, tamanhos 21..101	<code>sizes 21..101; (s+arange(w)) % nb</code>	Idêntico (default)
Size factor do pool = mediana(pool/ref)	<code>median(pooled / ref)</code>	Idêntico
Deconvolução: sistema linear $A \cdot \beta = R$	<code>lsqr(A, R)</code> na GPU	Mesma formulação
Reescala entre clusters (mediana pseudo)	<code>median(ps / ref_ps)</code>	Idêntico
Centra size factors em média 1	<code>sf /= sf.mean()</code>	Idêntico

Por que a GPU compensa aqui (≠ clustering): a normalização por bloco é cálculo pesado e regular — divisões, medianas sobre janelas deslizantes e um solve linear — sobre um bloco denso e limitado (≤ 3.000 células $\times \sim 14k$ genes ≈ 170 MB) que cabe na VRAM. Cada bloco vai à GPU uma vez, calcula muito, e volta. Não há os milhares de idas-e-vindas pela PCIe que afundaram o clustering. É um caso onde a GPU rende.

Divergências e seus trade-offs:

- **Soma dos pools por prefix-sum (cumsum):** o scran soma de forma incremental (sequencial, ótima em CPU); reescrevemos com prefix sums, vetorizáveis na GPU. Mesma soma, caminho casado ao hardware.
- **log natural (log1p) vs log2 do scater:** diferem só por uma constante ($\ln x = \log_2 x \cdot \ln 2$); irrelevante para clustering/PCA, mas não é bit-idêntico ao pipeline do paper.
- **Equações de tamanho 1 (identidade) adicionadas:** regularizam o sistema (cada célula ganha uma linha direta). O scran não faz por default — o menor pool dele é 21. É uma adição estabilizadora, defensável.
- **Fallback por equações normais:** o caminho primário é LSQR (como o scran); o fallback eleva ao quadrado o condicionamento, mitigado por ridge e pelo teto de 3.000 células.
- **Bloco denso + max_block=3.000:** abrimos mão da esparsidade dentro do bloco para ganhar vetorização densa, controlando a VRAM com o teto. Clusters maiores são fatiados e reescalados entre si pela mediana das pseudo-células.

Validação: a lógica foi conferida em NumPy contra size factors conhecidos — corr 0,99 (1 cluster) e 0,997 (multi-cluster), com erro $\sim 2,6$ –6% contra ~ 15 –23% da library-size simples. O algoritmo reproduz o scran; as divergências são de implementação, não de método.

7 — PCA: troca de algoritmo (IRLBA → covariância)

Como nas outras etapas, o paper **não descreve o PCA** — usa o **IRLBA** (Lanczos bidiagonal reiniciado) via BiocSingular, que levou **96 horas**. O IRLBA é um SVD **truncado e iterativo**: nunca

forma $X^T X$, trabalha por produtos matriz-vetor e **relê a matriz do disco a cada iteração**. É genérico — e o preço da generalidade é I/O: dezenas de passadas pelo disco = as 96 h. **O gargalo do paper aqui é leitura repetida, não cálculo.**

Nosso método (covariância / Gram): como **n (1,1M) $\gg$ p (13,9k genes)**, a matriz de covariância $\text{gene} \times \text{gene}$ é pequena (~1,5 GB) e cabe na VRAM. O PCA sai em **exatamente 2 passadas** pelos dados:

- **Passe 1:** acumula $S = X^T X$ por batch + somas de coluna (média). $M = X^T X - n \cdot \mu \mu^T$ centra **sem densificar** X .
- **eigh(M):** top-50 autovetores = loadings, resolvido direto na VRAM, **zero releitura**.
- **Passe 2:** projeta scores = $(X - \mu) \cdot V$ em streaming.

É **matematicamente exato** (não aproximado): a eigendecomposição de $(X - \mu)^T (X - \mu)$ dá os mesmos componentes que o SVD de $X - \mu$. Validado: **corr 1,000000 vs sklearn PCA** nos 50 PCs.

Insight central (frase de banca): a maior parte do speedup de 96 h $\rightarrow$ minutos é ALGORÍTMICA, não da GPU. Reconhecemos que $n \gg p$ e trocamos SVD iterativo out-of-core por covariância em 2 passadas; mesmo em CPU isso esmagaria o IRLBA. A GPU é multiplicador (acelera o `eigh` e os `matmuls`), não a fonte do ganho.

Trade-offs:

- **Condicionamento:** formar $X^T X$ **eleva ao quadrado** o número de condição — pior que SVD direto nos PCs de baixa variância. Mitigado: acumulação e `eigh` em **float64**, só top-50 PCs (os mais robustos), e validação contra sklearn.
- **Precondição de memória:** precisa do $p \times p$ na VRAM; só vale quando p é pequeno. O IRLBA funciona para p arbitrário. É **especialização vs generalidade** — nossa solução é ótima para este dataset, não geral.
- **Centralização sem densificar:** a identidade $M = X^T X - n \cdot \mu \mu^T$ mantém X esparsa (subtrair μ de uma matriz esparsa destruiria todos os zeros).
- **Só centralização, sem z-score:** bate com o default do `scater` (`scale=FALSE`).

8 — Frases-âncora: normalização e PCA

Sobre a normalização:

“A normalização não é do paper — ele usa o `scrn`. Reimplementei o algoritmo de deconvolution do `scrn` na GPU, fiel passo a passo (pré-escala, pools circulares 21-101, mediana de razões, deconvolução linear, reescala entre clusters), e validei contra os size factors de referência. A GPU compensa aqui porque o cálculo por bloco é pesado e regular, sobre um bloco denso que cabe na VRAM.”

Sobre por que `scrn` e não normalização simples:

“Normalizar por library size assume que toda célula tem a mesma composição de RNA. O `scrn` agrupa células parecidas e normaliza por pooling, respeitando diferenças biológicas reais — por isso depende de um clustering inicial.”

Sobre o PCA:

“O paper gastou 96 h com IRLBA porque ele relê a matriz do disco a cada iteração. Como $n \gg p$, troquei por covariância $\text{gene} \times \text{gene}$ + `eigh` em 2 passadas — matematicamente idêntico (corr 1,0 vs sklearn). O ganho é principalmente algorítmico; a GPU acelera por cima.”

Sobre o trade-off do PCA:

“Formar $X^T X$ eleva ao quadrado o condicionamento, então usei `float64` e só os top-50 PCs. E o método só funciona porque p (genes) cabe numa eigendecomposição na VRAM — é uma solução especializada para este dataset, não geral como o IRLBA.”

9 — Clusterização final: K-Means completo via Lloyd, pós-PCA

Diferente da normalização e do PCA — que o paper terceiriza —, **a clusterização final é um resultado central do paper**: é ela que produz os 15 clusters da Fig. 5, validados por marcadores biológicos. Aqui existe um método do paper de verdade para comparar, e o nosso código não só reproduz: vai além, operacionalizando uma sugestão que o próprio paper faz mas nunca executa.

O que o paper fez: depois do PCA para 50 PCs, mbkmeans in-memory, **K=15 fixo**, em **~30 segundos**. O paper afirma textualmente que esse tempo seria “rápido o bastante para rerodar com número diferente de clusters, ou em subamostras para análise de estabilidade” — mas escolhe um K e para. Essa frase, nunca executada pelos autores, é exatamente o que nosso código faz.

Por que Lloyd completo, e não mini-batch (de novo):

	Clustering inicial (passo 4)	Clustering final (este passo)
Dado	1,1M × 13,9k, esparso, não cabe na VRAM	1,1M × 50, denso, cabe (220 MB)
Onde mora	Streamado pela PCIe, batch a batch	Residente na VRAM
Algoritmo certo	Mini-batch (obrigatório por memória)	Lloyd completo (viável e mais exato)
Gargalo	Transferência PCIe repetida + esparsidade	Nenhum — é cálculo denso
GPU compensa?	Não (data starvation)	Sim, decisivamente

A razão de usar Lloyd e não mini-batch está toda aí: o mini-batch existe para evitar carregar tudo na memória. Mas os 50 PCs cabem inteiros na VRAM, então não há mais motivo de memória para aproximar — Lloyd completo usa **todos** os pontos a cada iteração, e cada iteração é **um único GEMM** ($n \times d$) · ($d \times k$), o padrão onde a GPU é imbatível. O detalhe honesto: o próprio paper mostra que mini-batch $\approx$ k-means exato em acurácia (batch ≥ 500), então o upgrade para Lloyd não muda as conclusões biológicas — só remove a aproximação. É um ganho de exatidão de graça, possível só porque o dado ficou pequeno e denso.

Nossa contribuição além do paper: busca sistemática sobre múltiplos valores de K (não um K fixo), com **n_init=3** restarts por K para evitar mínimos locais, e seleção de K orientada por evidência quantitativa: WCSS/n (método do cotovelo) e silhouette amostrado, em vez de um valor escolhido a priori.

Resultados observados e o que eles revelam sobre o dado: a curva de WCSS/n decresce de forma suave ao longo de K, sem um cotovelo forte e bem definido. O silhouette amostrado fica uniformemente baixo (0,00–0,03) para praticamente todos os K testados, com um pico isolado em K=2 (~0,20) seguido de degradação rápida — padrão consistente entre duas rodadas independentes de teste, com os valores de silhouette batendo nos K's em comum entre elas (evidência de que é sinal real, não ruído de execução).

- Por convenção, silhouette < 0,25 indica estrutura de cluster fraca ou ausente — os clusters estão se sobrepondo no espaço de PCs.
- **Hipótese:** dado de neurônios frequentemente forma um contínuo de diferenciação/maturação, não clusters discretos e bem separados. K-Means sobre distância Euclidiana assume clusters globulares — quando a estrutura real é um gradiente, qualquer K dá silhouette baixo porque não existem fronteiras nítidas para encontrar. É por isso que a literatura moderna de scRNA-seq frequentemente prefere clustering baseado em grafo (Louvain/Leiden sobre KNN) a K-Means para esse tipo de estrutura.
- Isso **não invalida** a implementação — é uma limitação conhecida do método (K-Means/Euclidiana) frente ao regime do dado, não um bug de engenharia.

Validação de integridade do pipeline: como parte do rigor metodológico, construímos um validador que percorre raw → filtrado → clusterizado-inicial → normalizado → PCA em blocos (streaming, sem nunca materializar a matriz inteira na RAM) e confere consistência entre etapas — contagem de células, conjunto de genes, barcodes duplicados, e a checagem mais crítica: que o PCA corresponde exatamente ao dado normalizado atual. Todas as checagens passaram (1.148.558 células em todas as etapas, 0 barcodes duplicados, 13.897 genes consistentes). Isso reforça que o silhouette baixo é estrutura real do dado, não desalinhamento entre etapas do pipeline.

Divergências e seus trade-offs:

- **Streams + threads:** o ganho de concorrência é real mas depende da carga — para K grande/n grande, o GEMM já satura a GPU sozinho, então streams concorrentes têm pouco o que sobrepor; o overlap rende mais em K's pequenos.
- **VRAM escala com K's concorrentes:** cada thread aloca seu próprio D_mat ($n \times k$); mitigado no v4 limitando quantas threads rodam ao mesmo tempo.
- **Elbow com poucos pontos é ruidoso:** a 2ª derivada discreta sobre poucos valores de K amplifica ruído; mitigado usando o eixo de K real (não índice uniforme) e, quando disponível, o algoritmo Kneedle (biblioteca kneed).
- **Silhouette é uma adição nossa, e é estimativa:** o paper usa WCSS e ARI, não silhouette. A silhouette amostrada é um estimador com ruído de amostragem — deve ser apresentada como estimativa, não valor exato.
- **Inconsistência menor no retorno do Lloyd:** o WCSS e os labels retornados vêm dos centróides de antes do último update, não do centróide final retornado. Na convergência a diferença é desprezível, mas é tecnicamente um passo de atraso — vale citar como limitação conhecida, não como erro oculto.

Comparação justa de tempo: o paper roda 1 K-fit em ~30 s. Nosso código roda múltiplos K's $\times$ n_init restarts — comparar o tempo TOTAL contra os 30 s não é justo. A métrica comparável é o tempo médio por K-fit. [tempo a inserir após a medição final da v4] — esse é o número que sustenta a comparação na banca, não o tempo total do experimento.

10 — Frases-âncora: clusterização final

Sobre por que Lloyd completo:

“O clustering inicial precisa de mini-batch porque o dado esparsa não cabe na VRAM. Depois do PCA, a matriz tem só 50 dimensões e cabe inteira — 220 MB. Não há mais motivo de memória para aproximar, então uso Lloyd completo: mais exato, e o próprio paper mostra que mini-batch já era equivalente a exato em acurácia, então não perco nada trocando.”

Sobre ir além do paper:

“O paper roda K=15 fixo e comenta que daria para rerodar com outros valores de K para análise de estabilidade — mas nunca faz isso. Meu código executa exatamente essa sugestão: busca sobre múltiplos K's, múltiplos restarts, e seleciona o K com evidência quantitativa (cotovelo + silhouette), não por escolha a priori.”

Sobre o silhouette baixo:

“O silhouette ficou baixo para praticamente todo K testado, e isso é consistente entre rodadas independentes — não é ruído. A hipótese mais provável é que dados de neurônios formam um contínuo de diferenciação, não clusters discretos, e K-Means sobre distância Euclidiana tem dificuldade conhecida nesse regime. Validei a integridade do pipeline ponta a ponta para descartar desalinhamento de dado como explicação alternativa.”